

GAN

➤ 관련 과목

 [보아즈](#)

Generative Adversarial Nets

Abstract

저자는 적대적 프로세스(= 경쟁)를 통해 생성 모델을 추정하기 위한 새로운 프레임워크를 제안하는데, 이 프로세스에서는 두 가지 모델을 동시에 훈련시킨다. 하나는 생성 모델 generative model G로, 데이터의 분포를 포착하며, 다른 하나는 discriminative model D로, 특정 샘플이 G에서 나온 것이 아니라 훈련 데이터에서 왔을 확률을 추정한다. G를 훈련하는 절차는 **D가 실수를 할 확률**, 즉 D가 G가 만든 가짜 샘플을 실수로 진짜라고 판단할 확률을 **최대화**하는 것이다. 이 프레임워크는 미니맥스 2인 게임과 같다.

GAN에서는 D는 분류 정확도를 최대화하려 하고, G는 D의 실수율(G의 성공률)을 최대화하려고 하는데, 이는 D의 관점에서 보면 G가 D의 손실(실수)을 최대화하려는 것과 같다. 따라서 D는 손실을 '최소(mini)'화하려 하고, G는 D의 손실을 '최대(max)'화하려 하므로 '미니맥스' 게임이 된다.

모델의 용량이 무한하다고 가정할 때 임의의 함수 G와 D의 공간에서 고유한 해가 존재하는데, 이때 G는 훈련 데이터 분포를 **완벽하게 복원**하고 D는 모든 곳에서 $\frac{1}{2}$ 과 **같아진다**.

G와 D가 다층 퍼셉트론으로 정의되는 경우, 전체 시스템은 역전파를 사용하여 훈련될 수 있다. 훈련 중이든, 또는 샘플을 생성할 때든, 마르코프 연쇄나 펼쳐진 근사 추론 네트워크와 같은 것이 전혀 필요 없다. GAN은 생성 과정이 단순한 순방향 전달이므로, 기존 생성 모델들이 겪던 주요 **계산상의 어려움**을 회피한다는 핵심적인 장점을 강조한다.

Introduction

딥러닝의 최종 목표는 **데이터를 완전히 이해**해서, 진짜와 똑같은 **가상 데이터**를 만들거나 데이터를 분류할 수 있는 '똑똑한 모델'을 만드는 것이다. 즉, 인공지능 분야에서 다뤄지는 데이터(예: 자연 이미지, 음성 파형, 자연어의 기호 등)의 생성 원리, **확률 분포를 나타낸다**. 이 모델은 정보를 계층적으로 쌓아 올려서 복잡한 데이터를 처리한다.

지금까지 딥러닝에서 가장 눈부신 성공을 거둔 것은 판별 모델이었는데, 이 모델들은 보통 복잡한 입력(이미지 등)을 하나의 분류 라벨로 매핑한다. Deep generative models은 상대적으로 영향력이 적었는데, 이는 최대 우도 추정과 관련된 많은 **다루기 힘든 확률 계산**을 근사하는 것의 어려움과, 생성 작업에서 조각별 선형 유닛의 장점을 활용하기 어려웠기 때문이다.

"복잡한 수학 계산이 문제라고? 그럼 그 복잡한 계산을 **아예 안 하는** 새로운 방법을 만들자!"



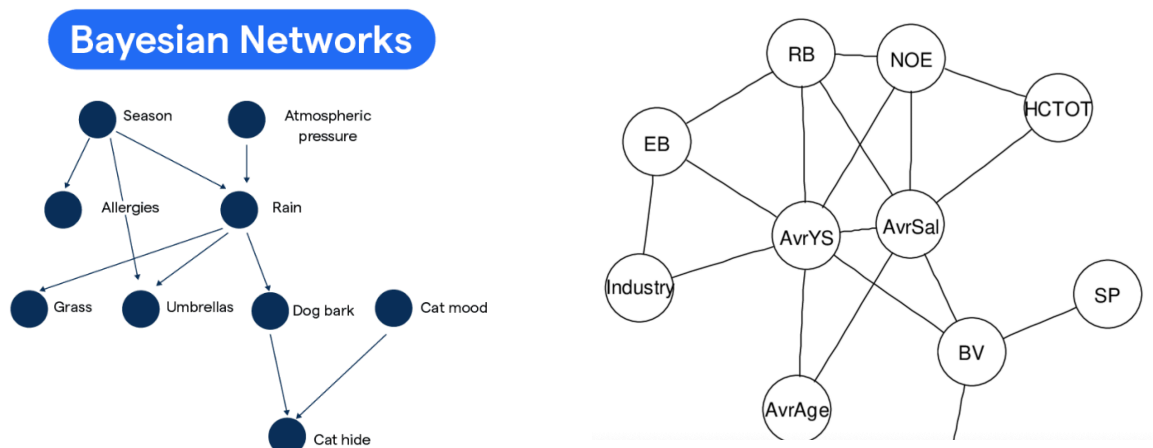
GAN의 핵심 아이디어: 위조지폐범과 경찰 (Adversarial Nets)

제안된 **적대적 신경망** 프레임워크에서는 생성 모델(G)이 **적**, 즉 특정 샘플이 모델에서 나온 것인지 아니면 실제 데이터에서 나온 것인지를 구별하도록 학습하는 판별 모델(D)과 **경쟁**하도록 한다. 생성 모델은 **위조지폐범** 팀에 비유할 수 있는데, 이들은 위조 화폐를 만들어서 들키지 않고 사용하려고 한다. 반면에 판별 모델은 **경찰**에 비유할 수 있으며, 위조 화폐를 탐지하려고 노력한다. 이 게임에서의 경쟁은 두 팀(G와 D) 모두가 **가짜가 진짜와 구별 불가능해질 때까지** 그들의 방법을 개선하도록 만든다. G가 더 정교한 가짜를 만들면, D도 더 날카로운 감별법을 익혀야 한다. 이 경쟁이 계속되면, 결국 G는 **진짜 데이터와 완벽히 똑같은** 가짜를 만들어내게 되고, D는 더 이상 진짜와 가짜를 구별할 수 없게 된다. 이 **적대적 경쟁**을 통해 생성 모델 G는 **최고의 성능**에 도달하게 된다.

이 논문에서는 생성 모델이 다층 퍼셉트론을 통해 랜덤 노이즈를 통과시켜 샘플을 생성하고, 판별 모델 역시 다층 퍼셉트론인 특별한 경우를 다룬다. 저자는 이 특별한 경우를 적대적 신경망이라고 부른다. 이 경우, 우리는 매우 성공적인 역전파와 **드롭아웃** 알고리즘만을 사용하

여 두 모델을 모두 훈련시킬 수 있으며, 생성 모델로부터 샘플을 만들 때도 단순히 순방향 전파만을 사용한다.

Related work



잠재 변수가 있는 Bayesian Network 같은 **방향성 그래프 모델**의 대안으로, RBMs이나 **DBMs** 같은 **무방향성 그래프 모델**들이 있다. 이 모델들은 데이터 요소들 간의 관계를 **방향 없이** 본다. 이 모델들 내부의 상호작용은 '정규화되지 않은 잠재 함수'들의 곱으로 표현되는데, 이는 모든 확률 변수의 상태에 대한 **전역적인 합산이나 적분**을 통해 정규화된다. 이 **파티션 함수**와 그 기울기는 가장 단순한 경우를 제외하고는 다루기 불가능 하기 때문에 **마르코프 연쇄 몬테카를로** 방법을 통해 **추정**할 수는 있다.



파티션 함수란?

확률 모델은 항상 "확률의 합 = 1"이 되어야 한다.

그런데 고차원 데이터(예: 1000픽셀 이미지, 수만 차원의 벡터)는

- 모든 경우의 수를 다 합쳐서 1로 정규화하기가 너무 힘들다는 단점이 있음.
- 분포의 모양도 미리 알 수 없음.

→ 확률을 직접 다루기에는 너무 복잡.

에너지는 단순한 "점수" 개념

확률 대신 "그럴듯함"을 점수화 → 낮을수록 자연스러운 데이터

에너지는 0 이상이면 되고, 합쳐서 1이 될 필요 없음.

즉, "확률처럼 복잡한 제약 조건"이 없는, 단순한 표현 방식.

에너지 → 확률로 바꾸기 쉽다

- 나중에 확률이 필요하다면, 에너지에서 **Softmax 비슷한 변환**을 하면 돼요:

$$P(x) = \frac{e^{-E(x)}}{Z}$$

- $E(x)$ 낮으면 확률 ↑
- $E(x)$ 높으면 확률 ↓
- Z = 정규화 상수(파티션 함수), 전체 합이 1 되게 해줌

→ 이렇게 하면 "점수"를 바로 "확률"로 바꿀 수 있음.

근삿값을 구하기 위해 MCMC를 쓰는데, 이 방법은 샘플들이 데이터 공간을 충분히 돌아다녀야 정확해진다.

하지만 데이터가 복잡할수록 샘플들이 한 부분에 갇혀서 제대로 된 근삿값을 얻기 어렵다는 큰 문제가 있었다.

DBNs은 하나의 무방향 층과 여러 개의 방향 층을 포함하는 하이브리드 모델이다. 비록 빠르고 근사적인 층별 훈련 기준이 존재하긴 하지만, DBN은 무방향 모델과 방향 모델 양쪽의 계산상의 어려움을 모두 떠안게 된다. 로그-우도를 근사하거나 제한하지 않는 대안적인 학습 기준들로 스코어 매칭과 노이즈 대비 추정이 제안되기도 했다.

→ 계산 불가능한 확률 문제를 아예 피하려고, 확률을 직접 계산하지 않는 새로운 학습 방법(스코어 매칭, NCE)을 만들었다.

이 두 가지 방법은 학습된 확률 밀도가 정규화 상수를 제외하고는 수학적으로 명확하게 지정될 수 있어야 한다. DBN이나 DBM처럼 여러 층의 잠재 변수를 가진 흥미로운 생성 모델들에서는, 다루기 쉬운 비정규화 확률 밀도를 도출하는 것조차 불가능하다. NCE는 GAN과 비

숫하게, 데이터를 '구별하는 능력'을 이용해 생성 모델을 학습시키려 했다. GAN과 다르게 별도의 판별 모델을 학습시키는 대신, 생성 모델 자체가 스스로 생성된 데이터를 고정된 노이즈 분포의 샘플과 구별하는 데 사용되었다. NCE의 비교 대상인 노이즈는 고정되어 있다. 생성 모델이 조금만 좋아져도 노이즈와 구별하는 것이 너무 쉬워져서, 더 이상 개선되지 않고 학습이 정체되는 문제가 발생했다.

일부 기술들은 확률 분포를 명시적으로 정의하지 않고, 단지 원하는 분포에서 샘플을 추출하도록 생성 기계를 훈련시켰다. 이 분야의 대표적인 최근 연구로는 GSN 프레임워크가 있으며, 이는 매개변수화된 마르코프 연쇄를 정의하는 것으로 볼 수 있다. 즉, 생성 마르코프 연쇄의 한 단계를 수행하는 기계의 매개변수를 학습한다.

Adversarial nets

1. 모델 구조

적대적 모델링 프레임워크는 두 모델(G와 D)이 모두 다층 퍼셉트론(MLP)일 때 가장 쉽게 적용된다.

데이터 x 에 대한 생성자의 분포 p_g 를 학습하기 위해, 우리는 입력 노이즈 변수 z 에 대한 사전 분포 (G에게 주입하는 노이즈 z 의 초기 분포 = $p_z(z)$)를 정의한 다음, 이 노이즈를 데이터 공간으로 매핑하는 함수를 $G(z; \theta_g)$ 로 나타낸다. G는 매개변수 (G가 학습해야 할 가중치(매개변수)) θ_g 를 가진 **미분 가능한 MLP**이다.

저자는 또한 하나의 스칼라(단일 숫자)를 출력하는 두 번째 MLP $D(x; \theta_d)$ 를 정의한다. $D(x)$ 는 입력 x 가 **생성 모델 p_g 에서 나온 것이 아니라 실제 데이터에서 나왔을 확률**을 나타낸다. 즉 x 가 진짜일 확률(0과 1 사이의 값)을 나타내며 θ_d 는 D가 학습해야 할 가중치이다. D가 훈련 데이터(진짜)와 **G에서 나온 샘플** 모두에 **정확한 라벨**을 할당할 확률을 **최대화**하도록 훈련시킨다. 동시에 G는 $\log(1 - D(G(z)))$ 를 **최소화**하도록 훈련시킨다.

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))] - (1)$$

- $\max_D$ (**경찰 D의 목표**): D는 이 전체 식을 최대화하려 한다.
 - **앞 부분** ($\log D(x)$): 진짜 데이터(x)를 보고 "진짜($D(x) \approx 1$)"라고 할수록 값이 커진다.

- 뒷 부분 ($\log(1 - D(G(z)))$): 가짜 데이터($G(z)$)를 보고 "가짜($D(G(z)) \approx 0$)"라고 할수록 $1 - D \approx 1$ 이 되어 값이 커진다.
- $\min_G$ (위조지폐범 G의 목표): G는 D가 최대로 올린 이 값($\max_D V$)을 다시 최소화하려 한다.
 - G는 D가 자기 생성물을 '가짜'라고 판단하지 못하게 해서, 뒷부분($\log(1 - D(G(z)))$)을 작게 만들려 한다.
- 결론: D는 정확도를 올리려 하고, G는 D의 정확도를 낮추려 하는 **제로섬 경쟁**을 이 수식이 나타낸다.

훈련할 때마다 '경찰 D'가 완벽해질 때까지 기다렸다가 '위조범 G'를 훈련시키는 것은 시간이 너무 오래 걸리고, D가 학습 데이터만 외우는(과적합) 문제가 발생한다.

대신, 우리는 D를 k 번 **최적화**하는 단계와 G를 **1번 최적화**하는 단계를 **교대로** 수행한다. G가 충분히 느리게 변하는 한, 이 전략은 D를 최적 해에 가깝게 유지한다.

• 실제 훈련 방식:

1. **경찰 D 훈련 (k 번):** D를 먼저 여러 번 훈련시켜서 구별 실력을 빠르게 올린다.
2. **위조범 G 훈련 (1번):** G를 한 번 훈련시켜서 더 좋은 가짜를 만든다.
3. 이 과정을 계속 반복한다. 이 방법은 D가 너무 강하지도, 너무 약하지도 않은 **적절한 수준**을 유지하도록 돕는다.

실제로, 수식 (1)은 G가 잘 학습하기에 충분한 기울기를 제공하지 않을 수 있다. 학습 초기에 G의 성능이 낮을 때, D는 생성된 샘플들이 훈련 데이터와 명확히 다르기 때문에 높은 확신을 갖는다. 이 경우, $\log(1 - D(G(z)))$ 값이 포화됩니다. G를 $\log(1 - D(G(z)))$ 를 최소화하도록 훈련하는 대신, G를 $\log D(G(z))$ 를 최대화하도록 훈련할 수 있다. 이 목표 함수는 G와 D의 역동성에서 동일한 종착점에 도달하게 하지만, 학습 초기에 훨씬 강한 기울기를 제공한다.

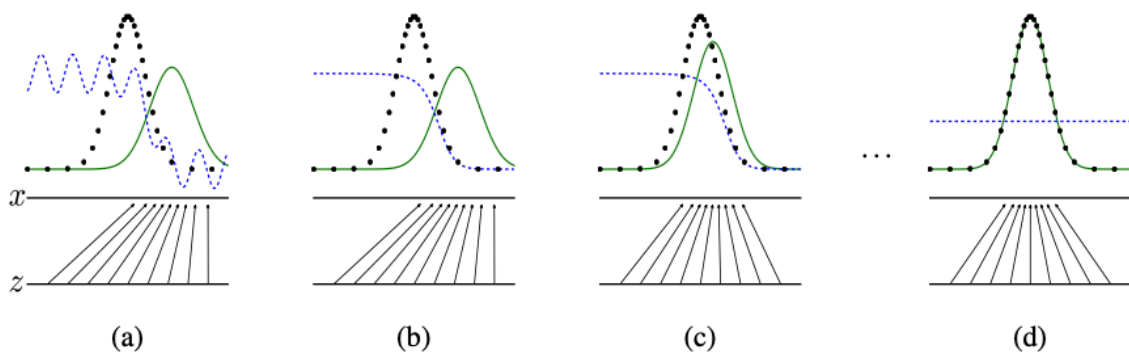


그림 1은 생성 분포 p_g (G, 녹색 실선)의 샘플과 데이터 생성 분포 p_x (검은색 점선)의 샘플을 구별하도록 판별 분포 D (파란색 점선)를 동시에 업데이트하며 GAN이 훈련되는 과정을 보여준다.

(a)는 수렴 근처의 상태를 보여준다. p_g 는 p_{data} 와 유사하고, D는 부분적으로만 정확한 분류기이다.

(b)는 알고리즘의 내부 루프에서 D가 데이터를 구별하도록 훈련되어 최적의 D에 수렴하는 모습을 보여줍니다.

(c)는 위조범(G)이 경찰(D)의 판단을 듣고 가짜 샘플(녹색 실선)을 진짜 샘플(검은 점선) 쪽으로 더 가깝게 이동시켜서 더 진짜처럼 보이게 만든 상태를 보여준다.

(d)는 최종 상태로, 가짜(녹색)와 진짜(검은색)의 분포가 완벽히 일치한다. 경찰(D, 파란색 점선)은 어떤 샘플을 봐도 50% 확률로 "진짜인지 가짜인지 모르겠다"고 판단한다. 이것이 GAN 학습의 이상적인 종착점이다.

Theoretical Results

저자는 GAN이 **이상적인 상태**에서 어떻게 작동하는지를 수학적으로 증명하기 위해 몇 가지 가정을 한다.

- **p_g (생성된 분포):** 생성자 G가 노이즈 z 를 받아서 만들어내는 데이터의 분포를 p_g 라고 정의한다.
- **Non-parametric setting:** 생성자 G와 판별자 D가 **모든 종류의 함수를 표현할 수 있는** (즉, 능력이 무한한) 이상적인 모델이라고 가정한다.

이 이상적인 경쟁 게임(미니맥스 게임)의 궁극적인 목표(최적의 해)는 **생성된 분포(p_g)가 실제 데이터 분포(p_{data})와 완벽하게 같아지는 것**임을 증명하는 것이다.

Algorithm 1: 실제 학습 절차

1. 반복: 학습 과정을 정해진 횟수만큼 반복한다.
2. 판별자 D 훈련 (K 번): 실제 데이터 샘플과 G가 만든 가짜 샘플을 섞어 D를 훈련한다.
D는 정확도를 높이는 방향(기울기를 오르는 방향)으로 업데이트된다.
3. 생성자 G 훈련 (1번): G가 만든 샘플에 대해 D가 "진짜"라고 착각하게 만들도록 G를 훈련한다.
G는 D의 정확도를 낮추는 방향, 즉 목표 함수를 최소화하는 방향(기울기를 내리는 방향)으로 업데이트된다.
4. 교차 훈련: 이 과정을 k 번의 D 훈련과 1번의 G 훈련을 교대로 반복하며 진행한다.

최적의 판별자 D

생성자 G가 **고정되어 있을 때**, 판별자 D가 도달할 수 있는 최적의 상태(D^*)는 위와 같은 수식으로 결정된다.

$$D_G^*(x) = p_{data}(x) / p_g(x) p_{data}(x)$$

- $D^*(x)$ 의 의미: 어떤 샘플 x 가 주어졌을 때, D는 x 가 진짜(p_{data})일 확률과 가짜(p_g)일 확률을 비교하여 **진짜일 확률이 더 높은 쪽으로 분류하는 것이 가장 좋다**.
- 만약 진짜 데이터 분포(p_{data})의 밀도가 가짜 데이터 분포(p_g)의 밀도보다 3배 높다면, D는 그 샘플을 진짜일 확률 $\frac{3}{3+1} = \frac{3}{4}$ 로 판단하는 것이 최적이라는 뜻이다.

$$C(G) = \max_D V(G, D) \dots$$

판별자 D가 최적의 상태(D^*)에 도달했다고 가정하고, D의 목표(최대화) 부분을 없앤 후, 생성자 G의 목표 함수($C(G)$)를 **오직 G의 분포(p_g)에 대한 함수**로 다시 정리할 수 있다.

최종 수렴 지점

G의 궁극적인 목표 함수($C(G)$)는 **생성된 분포(p_g)가 실제 데이터 분포(p_{data})와 완벽하게 같아질 때** 비로소 최소값($-\log 4$)에 도달한다.

JSD가 0이라는 것은 두 분포가 **완벽하게 같다**는 의미이다.

결론: 따라서, GAN이 학습을 통해 $C(G)$ 를 최소화하려고 노력하는 과정은 **생성 분포(p_g)와 실제 분포(p_{data}) 사이의 차이(JSD)를 최소화하는 것과 정확히 같다.**

- **JSD:** 두 확률 분포(p_{data} 와 p_g)가 **얼마나 다른지**를 측정하는 지표이다.
JSD가 0이라는 것은 두 분포가 **완벽하게 같다**는 의미이다.
- **최소 조건:** $C(G)$ 가 최소값($-\log 4$)이 되려면, **JSD가 0**이어야만 합니다.

결론: 따라서, GAN이 학습을 통해 $C(G)$ 를 최소화하려고 노력하는 과정은 **생성 분포(p_g)와 실제 분포(p_{data}) 사이의 차이(JSD)를 최소화하는 것과 정확히 같다.**

최종 결론: JSD는 0보다 작을 수 없으므로, $C(G)$ 가 가질 수 있는 가장 낮은 값은 JSD가 0일 때($-\log 4$)이며, 이는 **G가 실제 데이터를 완벽하게 복제할 때만** 가능하다는 것이 증명된다.